NestSelf authentication and authorization: quick guide

The ten-minute version: what the auth platform does, the decisions behind it and where to look. The detailed walkthrough is `implementation.md`; the full specification with every requirement is `docs/tasks/auth/auth-strategy.md`.

What it does

NestSelf serves five populations from one platform: employees of client companies, their HR and managers, independent experts, NestSelf staff and external suppliers. Each person has exactly one identity for life, may belong to several employers over time, and sees only what their current role allows. Authentication answers "who is this"; authorization answers "what may they do right now, in this organization".

The decisions

Decision	What we chose	Why
Identity provider	Amazon Cognito, one user pool per environment in Frankfurt (eu-central-1)	Email one-time codes, authenticator apps and passkeys without building them; free at pilot scale; EU data residency. Behind an interface so it can be replaced.
Sign-in methods	Employees use an email code. Experts and staff use a password plus an authenticator app. Employers may later bring their own SSO.	Removes passwords for the largest population; no SMS costs in Serbia and Hungary.
Web sessions	The browser gets an opaque cookie; the API keeps Cognito tokens encrypted in Redis (Backend-for-Frontend)	Tokens never reach JavaScript; revoking a session is deleting a key; CSRF handled with a double-submit token.
Mobile	The app talks to Cognito directly (PKCE) and calls the API with a bearer token	The standard native pattern; tokens live in the device's secure store.
Authorization	Hand-built grants evaluated by a pure `authorize()` function	Delegation, spending ceilings, effective-access previews and a 140-case golden test table are native; tokens carry no roles, so access changes apply within seconds.
Organization structure	A tree of org units with a closure table	Cheap ancestor queries for "manager of this team and everything under it".
Database access	Drizzle ORM with PostgreSQL row-level security	Tenant isolation lives in the database: a query without an organization filter returns nothing. Partial indexes, checks and views are part of the schema.
Audit	Append-only, partitioned `audit_events` with retention tiers	Every consequential action is recorded; nothing sensitive (tokens, message bodies, raw IPs) ever is.
Background work	BullMQ on Redis, run by a separate worker process	Nightly reconciliation (expired grants, stale mobile token families, audit partitions) is audited like any user action and never runs inside the API process.
Enforcement	Every route declares `@Public`, `@Authenticated` or `@Authorize`; a CI test fails otherwise	Default deny cannot be forgotten.

Decision	What we chose	Why
Rollout	Cognito first, platform stacks later, mobile last	Local development stays local; the CDK app deploys only the auth stack unless asked for more.

How a request works

1. The browser sends the session cookie (or the app sends a bearer token).
2. `AuthenticationGuard` turns it into a `Principal`: who they are, their memberships, their active context.
3. `CsrfGuard` checks the CSRF header on mutating browser requests.
4. `AuthorizationGuard` reads the route's `@Authorize(action, resolver)`, loads grants and assignments (cached in Redis for 30 to 60 seconds), and asks `authorize()`. Denied requests get 404 when the caller should not even know the record exists, 403 otherwise.
5. The handler runs its writes inside an organization-scoped transaction; PostgreSQL row-level security blocks anything outside that organization.
6. Consequential actions are written to the audit log; access changes publish an invalidation that drops caches and disconnects affected WebSocket clients.

How sign-in works

- **Employee**: enter email → receive an **eight digit** code (Cognito's length, not configurable) → enter it → session cookie. Five wrong codes lock the address for 15 minutes, and one wrong code already ends that challenge: the token is single-use, so a retry answers `invalid_challenge` and the user must request a new code (`implementation.md`, sections 6 and 21).
- **Expert or staff**: enter email → password → authenticator code (first time: scan a QR code) → session cookie. Invited staff set a password first.
- **Invitation**: HR invites by email; the link previews the organization name and activates the membership on acceptance, then runs the matching sign-in.
- **Mobile**: email code directly against Cognito; the app refreshes tokens itself and uses short-lived tickets to open WebSockets.

Where to look

Question	Place
Which routes exist and which decorator they carry	`apps/api/docs/auth-core.md`
The guards and sign-in services	`apps/api/src/auth/`, `apps/api/src/authz/`
The decision engine and its golden table	`apps/api/src/authz/authorize.ts`, `golden-table.json`
Actions, templates, shared schemas	`packages/contracts/src/`
Database schema, migrations, seed	`apps/api/src/db/schema/`, `apps/api/drizzle/`, `apps/api/src/db/seed/`

Question	Place
Sessions and Redis keys	`apps/api/src/session/`, `apps/api/src/redis/redis-keys.ts`
Sign-in screens	`apps/workspace/src/auth/screens/`, `apps/admin/src/auth/screens/`, `apps/mobile/src/auth/`
Cognito infrastructure and rollout steps	`CDK-setup.md` (repo root), `infra/cdk/lib/auth-stack.ts`, `infra/cdk/README.md`
Nightly jobs and the worker process	`apps/api/src/jobs/`, `apps/api/src/worker/main.ts`
Phase 2 plan	`docs/tasks/auth/phase-2-readiness.md`
Environment variables	`docs/environment.md`, `apps/api/src/config/env.ts`

Running it locally

```
pnpm install && pnpm db:up
cp apps/api/.env.example apps/api/.env
pnpm --filter @nestself/api db:migrate && pnpm --filter @nestself/api db:seed
pnpm --filter @nestself/api dev
pnpm --filter @nestself/api worker:dev   # optional: nightly jobs, needs REDIS_URL
```

Stub mode uses a fake identity provider pre-loaded with the sample people. Employees (`user_jovan@example.com`, `user_ivana@example.com`, `user_stefan@example.com`, `user_dragan@example.com`) sign in with the code printed in the API log; staff and experts (`user_nikola@example.com`, `user_milica@example.com`) use the password `Nestself-dev-1234!` and enrol an authenticator on first sign-in. Set `VITE_DATA_SOURCE=api` in a web app's `.env.local` to use the real screens. Skip `pnpm db:up` when PostgreSQL and Redis already run outside Docker; the Compose file claims port 6379 and fails against a local Redis or Valkey.

A real Cognito dev pool exists since 2026-10-24 (`eu-central-1_g6OGYONQ0` in account `634972095709`). Set `AUTH_MODE=cognito` with the values `infra/cdk/scripts/env-from-outputs.mjs dev` prints, and note that pool usernames are email addresses, not handles. `implementation.md` section 17.1 has the deployed coordinates and the three traps worth knowing before touching it.

Glossary

- **Principal**: the resolved identity for a request (user, memberships, active context, how they authenticated).
- **Context**: the hat the user wears right now: personal, a specific organization, expert, or operations.
- **Grant**: a user's permission to perform actions within a scope, with conditions and a validity window.
- **Template**: a named starting set of actions (`hr_admin`, `team_manager`, `employee`, ...) grants are created from.
- **Assignment**: a link to a work item (case, queue, client) that confers access without a grant.
- **Closure table**: every ancestor to descendant pair of the org tree, so scope checks are a set lookup.
- **Step-up**: re-authenticating recently before a sensitive action (not yet enforced).

What is not done yet

Step-up enforcement, grant administration screens, operations routes for assigned clients, emergency access, enterprise SSO, the remaining nightly jobs (session drift, Cognito user export, retention sweeper, audit export) and the compliance track (privacy notices, processing agreement, breach procedure), which must land before the first real employee signs in. The ordered plan is `docs/tasks/auth/phase-2-readiness.md`.